

Morse Code Converter

The Brief

A high-level but complete account of what the project is, how it is built, and where it is weak. Terms are defined in the glossary on the last page.

1 The project in one paragraph

A desktop application in Python that converts text to International Morse Code and back, live, as you type, in a two-tab window. It can play the current Morse code aloud as real 700 Hz beeps while flashing each dot and dash on screen in exact time with the sound. It is three Python files, roughly 600 lines including comments, and it has **no third-party dependencies**: the interface, the sound generation and the playback all run on Python's standard library alone. There is no database, no network access, and no configuration.

It began as a practice exercise in dictionary lookups and grew into an application. That history is visible in the code, and mostly in a good way.

2 Architecture

The organising decision is that **only one of the three files knows a screen exists**. The conversion logic and the audio logic can each be imported and used with no window ever opening, which is what makes the logic testable and let the audio be rewritten without touching the converter.

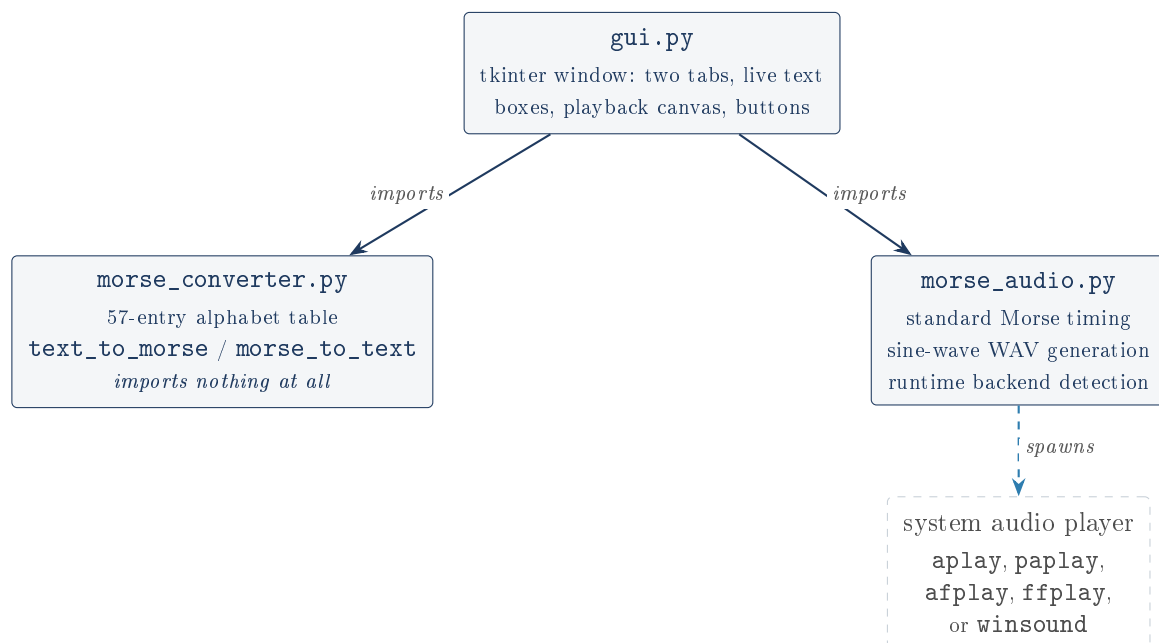


Figure 1: Module dependencies. Nothing points upward, and the two lower modules do not import each other in any real code path.

2.1 morse_converter.py: the alphabet

A hardcoded dictionary maps 57 characters (26 letters, 10 digits, the space, and 20 punctuation marks) to their Morse patterns. A one-line dictionary comprehension inverts that same table to

build the decoder, so there is exactly one place to edit a mapping. An excerpt, exactly as it appears in the source (the Deep Dive prints all 57 entries):

```

1 MORSE_CODE = {
2     "A": ".-.", "B": "-...", "C": "-.-.", "D": "-..", "E": ".",
3     # ... through I ...
4     "O": "---", "P": "-..", "Q": "-.-.", "R": ".-.", "S": "...",
5     # ... through Z, then the ten digits ...
6     "0": "-----", "1": ".----", "2": "--...", "3": "...--",
7     " ": "/",
8     "!": "-.-.-.", "?": ".-.-.-.", ".": "-.-.-.", ",": "-.-.-.",
9     # ... twenty punctuation marks in total, including "-": "-....-" ...
10 }
```

So `text_to_morse("SOS")` looks up S, O and S in turn and returns ... --- ..., the canonical Morse message: three dots, three dashes, three dots.

`text_to_morse` uppercases the input (this is the entire “case-insensitive” feature) and looks up each character, raising a `ValueError` that names the offending character rather than letting a low-level `KeyError` escape. `morse_to_text` reverses the process. Running the file directly still offers the original one-line command-line prompt, though it can only encode, not decode.

The row that does the quiet work

The space character is treated as an ordinary table entry whose Morse “pattern” is a forward slash (" ": "/"). Because `text_to_morse` joins per-character codes with single spaces, every word boundary in the output is literally the three characters " / ". That regularity is what makes decoding, and the playback timing, straightforward to parse.

2.2 morse_audio.py: real sound from nothing

Morse code is a pattern of *durations*, not just of symbols, and the standard fixes them all as multiples of one base unit (80 ms here):

dot	1 unit of tone	gap between letters: 3 units of silence
dash	3 units of tone	gap between words: 7 units of silence
gap inside a letter	1 unit of silence	

Table 1: The standard ratios, each a named constant in the source. Silence carries information: without three distinct gap lengths a listener cannot tell where one letter ends and the next begins.

`build_timing_sequence` turns a Morse string into an explicit ordered list of (`kind`, `units`) events, emitting gaps only *between* elements so a message never ends in trailing silence.

Sound itself is generated from first principles: `math.sin` produces a sine wave, `struct` packs the samples into raw bytes, and the standard library’s `wave` module writes them to a WAV file. This is the whole reason the dependency list is empty. The project’s own README records that it originally shipped visual-only playback because there “seemed to be no dependency-free way to make a sound”, and that this was simply wrong.

Playing the file is the harder half, because Python has no cross-platform way to reach a speaker. `detect_playback_backend` therefore **asks the machine rather than assuming**: it tries `winsound` on Windows, then checks for `aplay`, `paplay`, `afplay` and `ffplay` with `shutil.which` in that order, and treats “nothing found” as a legitimate answer. On this machine it returns `aplay`. If it returns nothing, the app says so plainly in the playback panel and keeps the visual flash working alone.

Tones are launched with `subprocess.Popen`, which starts the player and returns immediately rather than waiting for it to finish. This is load-bearing: if playback blocked for the 240 ms of a dash, the window would freeze and the picture would fall behind the sound.

2.3 gui.py: the window

A hand-styled dark tkinter interface. One factory function builds both tabs, handed a different conversion function each time, so identical machinery serves both directions. Each input box re-converts its entire contents on every `<KeyRelease>` event, which is why there is no submit button; conversion therefore runs constantly on invalid half-typed input, and a caught `ValueError` becomes a small inline message instead of a crash. Audio is detected and the two tone files generated at startup, before anything is drawn, so no delay is audible during playback.

3 The idea worth taking away

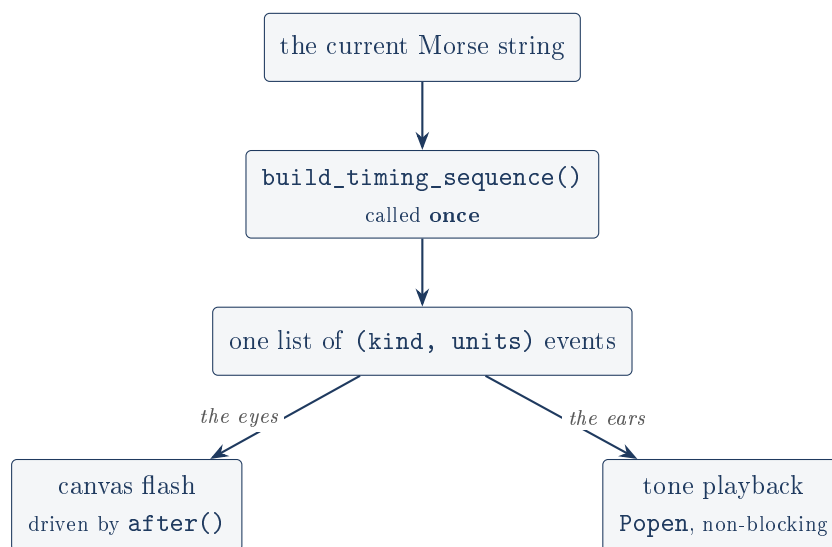


Figure 2: One timeline, two consumers.

Why this matters

The visual flash and the audio read from the **same** event list, so they cannot drift apart. The obvious alternative, hand-tuning a visual delay and an audio delay separately, is exactly what this project used to do, and the README records the result: a dot-to-dash ratio of roughly 1:1.9 instead of the correct 1:3, and one flat approximate gap where the standard requires three different ones. Deriving both outputs from a single source of truth is the fix, and it is the most transferable lesson in the project.

Two other decisions are worth naming. **Detecting the audio backend at runtime** rather than inferring it from the operating system means a machine with no player installed gets an honest message instead of a crash or a silent no-op. And **keeping the conversion functions free of any tkinter import** is what allowed the entire audio subsystem to be rewritten, twice, without the converter changing.

4 What I verified, and what I could not

Every behavioural claim below was checked by running the code, not read from the README:

- The 57-entry table inverts **losslessly**: no two characters share a Morse pattern, so the decoder loses nothing. Notably, nothing in the code *checks* this; it is correct by the table's contents, not by any guard.

- The round-trip property `morse_to_text(text_to_morse(s)) == s.upper()` held for every input tested: mixed case, digits, punctuation, a literal `/`, leading and trailing spaces, doubled spaces, all-whitespace.
- **Timing is exact.** Rendering "SOS" gave 27 units, and the resulting WAV measured 95,256 frames at 44,100 Hz, precisely 2160 ms, matching 27×80 ms computed by hand from the standard ratios. "SOS SOS" gave 61 units and 215,208 frames, exactly 4880 ms, so the 7-unit word gap is accounted for to the sample.
- `detect_playback_backend()` returns 'aplay' here, as documented.

I could not run the GUI (no display in this session), so statements about on-screen behaviour are read from the code. The `winsound`, `afplay` and `paplay` paths need Windows, macOS and PulseAudio respectively and were not exercised; the README states this limitation itself.

5 Honest assessment

The good is real. The module separation is genuine rather than nominal, the single event list is the right answer to a problem the project first got wrong, the runtime backend detection with a working fallback is more care than a practice project usually receives, and generating correct audio from `math.sin` and `wave` is a well-earned way to keep the dependency list empty. The README is unusually candid about the project's own missteps.

The weaknesses, in order of how much they matter:

No automated tests

The project's correctness lives almost entirely in a 57-entry table, and there is no test file. Every claim in the section above had to be re-established by hand. A silently wrong table entry is exactly the defect that manual spot-checks miss and a test would catch instantly. The README names this first among what it would do differently, which is the right instinct.

Play does not respect the tab you are looking at

`_current_tab_state()` is misnamed twice over: it returns a string, not a tab state, and it never asks which tab is selected. It reads the Text-to-Morse tab's output first and only falls back to the other tab. So typing in the first tab, switching to the second, typing there and pressing Play replays the *first* tab's content. The README acknowledges the fixed priority honestly, but the behaviour is still surprising.

Two contracts for the same malformed input

`morse_to_text` raises a `ValueError` naming a bad symbol, while `build_timing_sequence` silently discards anything that is not a dot or dash. I confirmed that `build_timing_sequence(".- XYZ -...")` drops XYZ without complaint yet still emits the gaps around it. The GUI masks this, but the two halves of the library genuinely disagree.

Redundant decoding logic, with a rationale that does not hold

`morse_to_text` handles word gaps twice: it splits the input on `" / "`, and `" / "` is *also* an ordinary decodable table entry meaning space. A naive one-line decoder that never splits on `" / "` produced identical output on every input I tested. The README defends the split by arguing that "splitting naively on `/` alone would have left stray spaces", but the alternative it overlooks is not splitting on the slash at all, and simply decoding it like any other symbol. Not a bug; redundant complexity with a stale justification.

Smaller items: `_safe_text_to_morse` and `_safe_morse_to_text` are pure pass-through wrappers whose `_safe_` prefix promises a safety they do not provide (the real error handling is elsewhere). The multi-line input boxes reject newlines, since `"\n"` is not in the table, so pressing Enter blanks the output with an inline error; it is handled gracefully but the widget invites input the alphabet cannot represent. `render_morse_to_wav` is unused by the application and is really a development instrument. And because tone processes are never collected, each finished player leaves a short-lived zombie process until the app exits.

None of these is a crash, a security issue, or a wrong answer. They are the ordinary residue of a script that grew into an application, and the distance between this code and its documentation is small.

Glossary

Morse code

Representing letters and digits as sequences of short signals (dots) and long signals (dashes), designed for telegraph wires where the only controllable thing was whether current flowed. “International” (ITU) Morse is the modern standardised form.

Dictionary (dict)

A container mapping keys to values, with instant lookup by key regardless of size, because it uses a hash table internally rather than scanning entries.

Module

A single Python file. One module can *import* another to use what is defined inside it.

GUI / tkinter / widget

A graphical user interface is the visible, clickable part of a program. *tkinter* is Python’s built-in GUI toolkit; a *widget* is any element within it (window, button, text box, canvas).

Event loop

The endless loop at a GUI’s heart that waits for something to happen (a key, a click, a timer) and calls the function registered for it. `root.after(ms, fn)` asks it to call `fn` later without blocking.

Callback

A function handed to something else to be called later, when an event occurs.

Exception / raising / ValueError

When a function cannot do its job it *raises* an exception, stopping immediately and handing an error up to its caller, who may *catch* it. `ValueError` conventionally means “this value is unusable”.

PCM / sample rate

Digital audio stores sound by measuring air pressure thousands of times per second. Each measurement is a *sample*; the rate here is 44,100 per second, the audio-CD rate. The raw list of numbers is *PCM*.

Sine wave / frequency / hertz

A pure tone is a value swinging smoothly up and down at a fixed rate. That rate, in hertz, is the frequency, perceived as pitch. This project uses 700 Hz.

WAV / the wave module

A WAV file is raw PCM samples with a small header declaring sample rate, channel count and sample size. Python’s standard library writes it, which is why no audio dependency is needed.

Process / subprocess.Popen / blocking

A process is a running program. `Popen` launches one and returns immediately; `run` and `wait` instead *block*, freezing the caller until it finishes.

shutil.which

Answers “would this command actually run on this machine, and where does it live?” by searching `PATH`. Returns `None` if not.

Standard library

The code that ships with Python itself, needing no installation. Everything this project uses comes from it.